

ML Weekend Project Plan

IRIS-style World Model + Custom Triton Kernels

Portfolio Project

3 Full Weekends

PyTorch + Triton

Cluster Required

Project Overview

Build an IRIS-style tokenized world model on Atari games, then profile and optimize its training loop with custom Triton kernels. The world model provides the *what* — a system worth optimizing — and the kernels provide the *how fast*. Together they demonstrate both deep architecture knowledge and systems engineering ability in one cohesive project.

What You Ship

- **A trained world model** that imagines future Atari game frames from actions alone — complete with GIF demos of imagined rollouts
- **A custom Triton kernel** for the training bottleneck (fused cross-entropy or VQ-VAE commitment loss), benchmarked against PyTorch baseline
- **A benchmarking suite** measuring speedup across batch sizes and sequence lengths on A100
- **A clean repo and README** with architecture diagram, training curves, and honest benchmark table

Architecture

Stage 1 — VQ-VAE Tokenizer

A Vector Quantized Variational Autoencoder compresses each 64×64 Atari frame into a short sequence of discrete integer tokens from a learned codebook (e.g. 16 tokens from a vocabulary of 512). This turns visual prediction into a language modelling problem. Watch for codebook collapse — many codes going unused — as the main training pitfall.

Stage 2 — Transformer World Model

A GPT-style transformer (~20M params) operates on token sequences interleaved with actions: [frame1_tokens, action1, frame2_tokens, action2, ...]. Trained autoregressively to predict the next token. At inference, feed it a real starting frame + action sequence and it hallucinates future frames entirely from imagination.

Data Flow

Real frame → VQ-VAE encoder → discrete tokens → Transformer predicts next tokens → VQ-VAE decoder → Imagined frame

Where the Kernels Live

Profile first — don't guess. The most common bottlenecks in this architecture are:

- **Fused cross-entropy** over codebook vocabulary — naively involves multiple memory round-trips (logits → softmax → log → loss). Fusing into one pass typically yields 3–5× speedup.
- **VQ-VAE commitment loss** — scatter + norm operation called every training step, clean fusion target.
- **Rotary embeddings (RoPE)** — if used in the transformer, fusing sin/cos application gives measurable gains at longer sequence lengths.

3-Weekend Plan

Each weekend assumes ~12–16 hours of focused work (full days, Saturday + Sunday).

Weekend	Day	Goal	End State
Weekend 1	Sat	Data pipeline: collect Atari frames via Gymnasium, save (frame, action, reward) tuples	100k+ frames on disk
	Sun	Train VQ-VAE tokenizer, verify reconstructions look coherent, check codebook utilization	Working tokenizer + reconstruction GIFs
Weekend 2	Sat	Train transformer world model on token sequences, generate first imagined rollouts	Imagined frames generating
	Sun	Profile training loop with torch.profiler, identify bottleneck, write correct (slow) Triton kernel	Correct kernel, not yet fast
Weekend 3	Sat	Optimize kernel until faster than PyTorch, run benchmark suite across batch/sequence configs	Benchmarks with speedup numbers
	Sun	Polish README, make figures, record final GIFs, clean code, push to GitHub	Shippable portfolio piece

Risk note: Weekend 2 has the highest variance. If the kernel you target turns out to be hard to beat (e.g. already well-optimized by cuDNN), pivot to a more tractable op. Profile first, choose your target based on data, not intuition.

Resume & Portfolio

Project Line

Tokenized World Model with Custom Triton Kernels (PyTorch, Triton) – github.com/you/repo

Bullet Points

- Trained an IRIS-style world model (VQ-VAE + 20M param transformer) on Atari frames, producing coherent multi-step imagined game rollouts
- Profiled training with torch.profiler and nsight, identified memory-bandwidth bottleneck, implemented fused Triton kernel for cross-entropy over discrete codebook vocabulary achieving X% wall-clock speedup over PyTorch baseline
- Benchmarked fused vs unfused training across batch sizes and sequence lengths on A100; documented results in reproducible benchmarking suite

Skills to List

Category	Skills
Frameworks	PyTorch, Triton, Gymnasium
Concepts	VQ-VAE, discrete latent representations, autoregressive transformers, model-based RL, kernel fusion
Systems	GPU profiling, mixed precision (bf16), SLURM, CUDA memory model, bandwidth-bound vs compute-bound ops
Tools	Weights & Biases, torch.profiler, NSight, torch.compile

Most important: The bullets matter less than the GitHub repo itself. A clean README with an architecture diagram, imagined rollout GIFs, and an honest benchmark table will do more for you than any resume line. Budget time for documentation — it is not wasted.

Background Knowledge

What is a Kernel?

A kernel is a function that runs in parallel on the GPU. Instead of writing a loop that processes one element at a time on the CPU, you write a function describing what one thread does, and the GPU runs thousands simultaneously. The mental shift: stop thinking about iterating over data, start thinking about what a single thread is responsible for.

Why Kernels Speed Things Up

Most training bottlenecks are memory-bandwidth bound — the GPU spends more time reading and writing memory than computing. Fused kernels reduce memory round-trips. Cross-entropy naively reads logits, writes softmax, reads that, writes log, reads that, writes loss — five memory passes. A

fused kernel does it all in one. That is why fused cross-entropy is often 3–5× faster.

Triton vs CUDA

- **Triton** — Python-like syntax from OpenAI. Think at the level of blocks of data, not individual threads. Handles low-level memory management for you. Recommended starting point.
- **CUDA (C++)** — Maximum control, much steeper curve. Worth learning eventually but not the right starting point here.
- **torch.compile** — Easiest entry point. Decorates a regular PyTorch function and fuses ops automatically. Good for understanding what fusion does before writing kernels manually.

Key Rule: Profile Before Writing

Never guess what the bottleneck is. Run your training loop under `torch.profiler`, open the trace in `chrome://tracing`, and find where time is actually spent. Ops that are compute-bound (large matrix multiplications, GEMMs) won't be beaten by a custom kernel — cuBLAS has been hand-tuned for years. Only target memory-bandwidth-bound ops.

IRIS (2023)

IRIS (Imagination with auto-Regression over an Inner Speech) from EPFL applied a discrete VQ-VAE tokenizer and GPT-style transformer world model to Atari. The RL agent trained entirely in imagination — never touching the real game environment during policy learning. Small, clean codebase, making it the ideal paper to replicate.

Key Challenges

- **Codebook collapse:** VQ-VAE codes go unused if commitment loss weighting is wrong. Monitor codebook utilization (% of codes used) as a training metric.
- **Compounding errors:** Transformer imagination drifts from reality over long horizons — expected behaviour, not a bug.
- **Kernel correctness:** A kernel that is faster but wrong is worse than no kernel. Always validate outputs against the PyTorch baseline before benchmarking.